PATTERN RECOGNITION | SEMESTER PROJECT | MONSOON 2026

Earthquake Building-Damage Grading

Feature extraction and class-balanced classical models for three-class damage prediction (Gorkha 2015, Nepal)

Team: Sri Harasha Adithya, KVNM Krishnavamsi, Kommi Vishal Venkat Vaibhav

Code repository: https://github.com/Krishnavamsi-codes/PR-Project-tbm

Dataset: Richter's Predictor (DrivenData / Kathmandu Living Labs), 5 November 2026

Abstract

We predict the damage grade (low, medium, high) of 260,601 buildings hit by the 2015 Gorkha earthquake from structural and geographic attributes. The problem is imbalanced (grade 1 is 9.6 %), so macro-F1 is our headline metric. We study how **feature extraction** (out-of-fold hierarchical target encoding of the geographic ids, engineered features, PCA, mutual-information and Fisher selection) and **modification of classical methods** (class-balanced training and nested-CV tuning) improve five model families: decision trees, random forests, SVMs, linear models and, as a deep-learning reference only, MLPs. All methods share one protocol: a fixed stratified 20,000-row subsample, stratified 5-fold CV repeated 3 times (15 runs), every learned step fitted on the training fold only, and Nadeau–Bengio corrected significance tests. Target encoding is the key step (macro-F1 +0.141 for the SVM, +0.113 for the MLP). A class-balanced random forest is best (0.658 ± 0.007 versus 0.581 for the best raw baseline), tied with a balanced linear SVM at one-sixth of the cost; PCA hurts and tuning helps only mis-specified models. On all 260,601 rows the balanced random forest reaches macro-F1 0.707 and accuracy 0.742, significantly ahead of the MLP (0.686), the balanced linear SVM and logistic regression.

1. Introduction

After the 2015 Gorkha earthquake, Nepal surveyed about 800,000 buildings and recorded for each its structure (age, floors, footprint, height, materials, foundation, roof), its location at three nested administrative levels and its **damage grade**: 1 (low), 2 (medium), 3 (near-complete destruction). Predicting the grade from cheap-to-record attributes helps rank buildings for inspection and shows which construction practices are most dangerous. It is a clean pattern-recognition problem: three classes, 38 mixed-type attributes, very high-cardinality identifiers and a skewed class distribution.

Always predicting grade 2 gives 56.9 % accuracy and finds no low-damage and no destroyed building. We therefore report **macro-F1** (unweighted mean of per-class F1) next to accuracy, with per-class precision/recall and confusion matrices. Following the course brief, our contribution is classical pattern recognition; deep learning is a comparison only. We ask: **RQ1** how much do feature-extraction steps improve classical classifiers, and which step matters? **RQ2** do class-balanced training and nested tuning improve them? **RQ3** which model family is best, and do the conclusions hold on all 260,601 rows? The study covers five families: decision tree, random forest, SVM (RBF, polynomial, sigmoid), linear models (logistic regression, linear SVM, ridge, SGD, perceptron) and MLPs.

2. Data and exploratory analysis

We use the labelled file of Richter's Predictor: **260,601 buildings, 38 attributes, no missing values**: 3 geographic ids (31 / 1,414 / 11,595 distinct values), 5 numeric (floors, age, area, height, families), 8 categorical (foundation, roof, floor types, plan, position, land surface, ownership; 38 one-hot columns) and 22 binary flags (11 superstructure materials, 11 secondary uses). This meets the course requirements (at least 10 attributes, 1,000 instances, 3 classes).

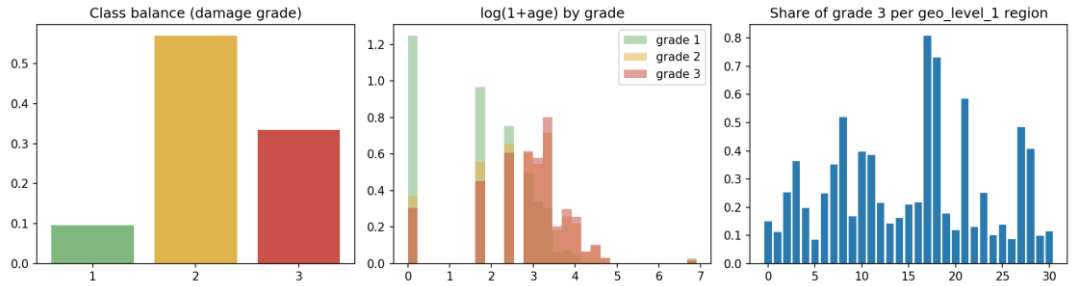


Figure 1. Class balance (left), log(1+age) by grade (middle) and share of grade-3 buildings per geo_level_1 region (right).

- **Imbalance:** grade 1 / 2 / 3 = 9.6 / 56.9 / 33.5 %, identical in the 20,000-row subsample.
- **Geography is a strong but nominal signal:** the grade-3 share per level-1 region ranges from 8.6 % (region 5) to 80.8 % (region 17). The ids are labels, not quantities, and the 11,595 level-3 ids (median 14 buildings, 27.6 % with five or fewer) cannot be one-hot encoded sensibly.
- **Structure matters:** 56.8 % of buildings on foundation type "i" are grade 1 and only 2.1 % grade 3 (37.8 % grade 3 on type "r"); engineered reinforced-concrete buildings are grade 3 in 1.8 % of cases against 34.0 % for the rest; younger buildings are over-represented in grade 1.

3. Experimental protocol

- **Data:** one fixed-seed (42) class-proportional subsample of **20,000** buildings for the main comparison (exact SVMs, forests and nested tuning scale poorly; the choice is ours, not a course rule). The methods that scale are repeated on **all 260,601 rows** (Section 6).
- **Resampling:** stratified 5-fold CV repeated 3 times (15 runs), identical splits for every method, so comparisons are paired. Mean ± std (n − 1) is reported; the full-data runs use 5-fold CV once (5 runs).
- **Metrics:** accuracy, macro-F1, per-class precision/recall/F1 and confusion matrices (summed over runs, row-normalised).
- **Significance:** paired t-test and the **Nadeau–Bengio corrected resampled t-test**, t = mean(d) / sqrt((1/k + n_test/n_train) var(d)), df = k − 1, because repeated-CV runs share data. We rely on corrected p-values; k = 15 (main) or 5 (full data).
- **No leakage:** every learned component lives in a scikit-learn Pipeline refitted on the training part of each fold (Table 1). Target encoding is out-of-fold inside the training fold.

Table 1. Learned components and where they are fitted ("training fold" = training part of the current outer split only).

Component	Fitted on	Safeguard
One-hot encoder, standard scaler	Training fold	Unseen levels map to zero; test data uses training statistics
Hierarchical target encoder (geo ids)	Training fold	Inner 5-fold out-of-fold for training rows; unseen regions fall back to the parent region
PCA, mutual-information / Fisher selection	Training fold	Refit in every fold
Class weights, SVM / forest / MLP parameters	Training fold	Class weights computed from training labels
Hyper-parameters (tuned models)	Inner 3-fold CV in each training fold	Nested CV: the outer test fold is never seen by the search

4. Methods

4.1 Feature extraction

Hierarchical smoothed target encoding. For region r at level l let n_r be the number of training buildings and n_{r,k} the number with grade k. The shrunk class probability is

$$p_l(k \mid r) = (n_{r,k} + \alpha \cdot p_{l-1}(k \mid \text{parent}(r))) / (n_r + \alpha), \quad \alpha = 20, \quad p_0 = \text{global class prior}$$

so a sparse level-3 region borrows from its level-2 and level-1 parents, and an unseen region falls back to its parent. We use P(grade 1) and P(grade 3) at each level (six dense features). Training rows are encoded **out-of-fold** (inner 5-fold) so that no row is encoded with its own label; test rows use maps fitted on the whole training fold (cf. Micci-Barreca, 2001). **Engineered features:** log age, age-unknown flag, area × height, area / height, floors × height, height per floor, counts of materials and secondary uses, reinforced-concrete and mud/stone flags. **Projection and selection:** PCA keeping 95 % of the variance, and top-20 filters by mutual information and by Fisher score, applied after encoding ("te_eng" = target encoding + engineered features).

4.2 Models

Decision tree (depth 10, 20 samples per leaf); **random forest** (300 trees, at least 2 samples per leaf); **SVM** with RBF (C = 1, gamma = "scale"), polynomial (degree 2, 3, coef0 = 1) and sigmoid kernels; **linear models**: softmax logistic regression (C = 0.3), linear SVM (C = 0.05), ridge classifier (α = 10), SGD (hinge, modified Huber; α = 0.001) and perceptron; **MLPs** (one to three hidden layers, ReLU / tanh / logistic, optional L2; early stopping), used **only as a deep-learning reference**. Features are standardised; categoricals are one-hot encoded. "Raw" means the geographic ids are passed as plain numbers; every other pipeline uses the target encoding.

4.3 Modifications of the classical methods

Class-balanced training. Each class receives weight w_k = n / (3 n_k) in the loss (class_weight = "balanced_subsample" for the forest and "balanced" for the SVMs, logistic regression and linear SVM). Because grade 1 is only 9.6 % of the data, the accuracy-optimal rule gives it a high threshold; balancing moves the boundary towards the rare class, which is what macro-F1 rewards. **Nested-CV tuning.** For eight models a 3-fold grid search (macro-F1) runs inside every outer training fold; the winner is refitted and scored on the outer test fold, so reported scores estimate "tune, then deploy". Grids: SVM C × gamma (RBF) or C × degree (polynomial), linear SVM and logistic regression C × class weight, tree depth × leaf size, forest depth × leaf × max-features, MLP width × alpha, Nystroem gamma × C.

5. Results on the 20,000-row subsample

All values are mean ± std over 15 runs; MF1 = macro-F1; Rec 1 = recall of the rare grade-1 class; s = seconds for the 15 runs.

5.1 Model families

Table 2. All five model families on the 20,000-row subsample (TE+eng = target encoding + engineered features unless "raw").

Model	Accuracy	Macro-F1	Rec 1	s
Decision tree, raw (best raw baseline)	0.661 ± 0.006	0.581 ± 0.009	0.370	1
Decision tree, TE+eng	0.688 ± 0.009	0.609 ± 0.013	0.388	4
Random forest	0.714 ± 0.005	0.640 ± 0.008	0.398	83
Random forest, balanced	**0.708 ± 0.006**	**0.658 ± 0.007**	**0.544**	**87**
SVM-RBF, raw	0.612 ± 0.004	0.477 ± 0.009	0.247	98
SVM-RBF, TE+eng	0.710 ± 0.005	0.621 ± 0.009	0.350	322
SVM-RBF, balanced	0.663 ± 0.005	0.633 ± 0.006	0.781	372
SVM polynomial degree 2	0.710 ± 0.005	0.626 ± 0.007	0.372	293
SVM sigmoid	0.613 ± 0.007	0.545 ± 0.011	0.417	201
Logistic regression	0.715 ± 0.006	0.641 ± 0.009	0.405	2
Logistic regression, balanced	0.671 ± 0.005	0.641 ± 0.006	0.770	2
Linear SVM	0.710 ± 0.005	0.629 ± 0.007	0.374	13
Linear SVM, balanced	**0.700 ± 0.004**	**0.656 ± 0.006**	**0.628**	**14**
Perceptron	0.634 ± 0.035	0.564 ± 0.031	0.429	1
MLP 128-64, raw (reference)	0.617 ± 0.007	0.522 ± 0.018	0.341	43
MLP 128-64, TE+eng (reference)	0.710 ± 0.004	0.635 ± 0.011	0.408	47
MLP 128-64, L2 (reference)	0.711 ± 0.004	0.643 ± 0.011	0.441	12

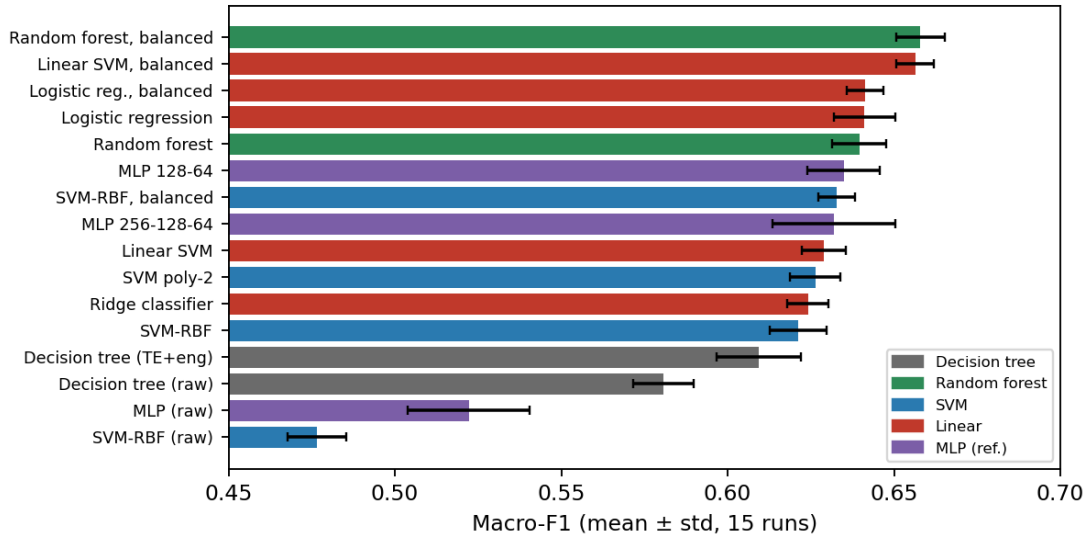


Figure 2. Macro-F1 (mean ± std over 15 runs) of the main models, coloured by family.

The best raw baseline is a decision tree (MF1 0.581). The **balanced random forest** is best overall (0.658 ± 0.007, +0.077, corrected p = <0.001). It is statistically tied with the **balanced linear SVM** (0.656; difference 0.002, p = 0.586), which needs 14 s instead of 87 s. Accuracy is almost flat across good models (0.70–0.715); the separation between them is in grade 1. Flexibility does not pay on 20,000 rows: the RBF-SVM (0.621), polynomial SVMs (≈0.625) and MLPs (0.632–0.643) are no better than plain logistic regression (0.641) and clearly below the balanced forest (p ≤ 0.002 against the balanced RBF-SVM and the MLP); the sigmoid kernel (0.545) and the perceptron (0.564) are unsuitable.

5.2 Feature extraction (RQ1)

Table 3. Ablation: macro-F1 as feature steps are added (steps after "+ TE + engineered" are applied on top of it; "–" = not run).

Feature set	SVM-RBF MF1	SVM-RBF acc.	Tree MF1	Tree acc.
raw ids	0.477	0.612	0.581	0.661
+ engineered features	0.488	0.612	–	–
+ target encoding	**0.618**	**0.709**	**0.610**	**0.690**
+ TE + engineered	0.621	0.710	0.609	0.688
+ MI top-20	0.627	0.709	0.613	0.689
+ Fisher top-20	0.630	0.709	0.613	0.688
+ PCA 95 %	0.610	0.700	0.588	0.672

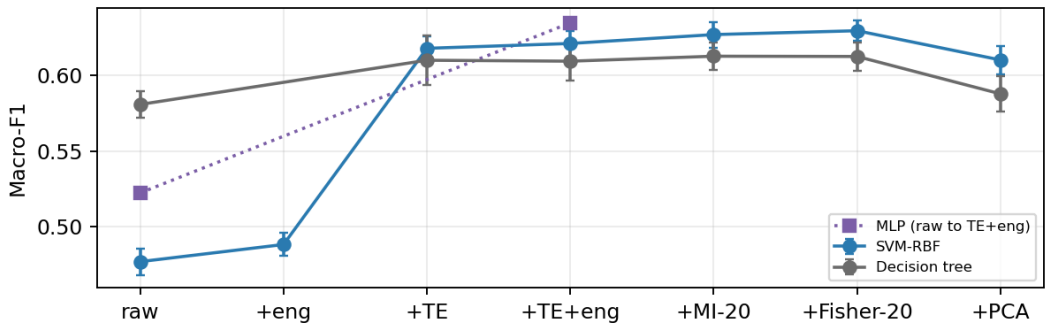


Figure 3. Ablation of the feature steps (macro-F1, mean ± std); the MLP is shown for raw versus TE+eng only.

Target encoding is the dominant step. Replacing the three raw ids by six out-of-fold regional probabilities raises macro-F1 by +0.141 for the SVM (accuracy 0.612 → 0.709), by +0.113 for the MLP (0.522 → 0.635) and by +0.029 for the tree (p = 0.008); the SVM and MLP use integer ids poorly, whereas a tree can split on them directly,

hence its smaller gain. Engineered features alone change the SVM by only +0.011 and add at most 0.003 on top of encoding. **Selection** gives small gains for the SVM (MI-20 +0.006, p = 0.141; Fisher-20 +0.008) and none for the tree. **PCA hurts** (SVM -0.011, p = <0.001; tree −0.022): it keeps directions of high variance, mostly one-hot and flag columns that say little about damage.

5.3 Class balancing and tuning (RQ2)

Table 4. Effect of class-balanced training (20,000 rows, 15 runs): plain → balanced.

Model	MF1 plain → bal.	Δ MF1	p (corr.)	Rec 1	Accuracy
Random forest	0.640 → 0.658	+0.018	<0.001	0.398 → 0.544	0.714 → 0.708
SVM-RBF	0.621 → 0.633	+0.012	0.035	0.350 → 0.781	0.710 → 0.663
Linear SVM	0.629 → 0.656	+0.027	<0.001	0.374 → 0.628	0.710 → 0.700
Logistic regression	0.641 → 0.641	+0.000	0.981	0.405 → 0.770	0.715 → 0.671

Balancing raises grade-1 recall in every model (e.g. SVM 0.35 → 0.78) at the cost of accuracy and of grade-1 precision, so it improves macro-F1 only where the gain in recall outweighs the loss in precision: significantly for the forest (+0.018) and the linear SVM (+0.027), marginally for the SVM-RBF (+0.012, p = 0.035), and not at all for logistic regression (+0.000), whose grade-1 precision drops as much as recall rises. The forest is the only model that gains macro-F1 while losing under 1 point of accuracy (0.714 → 0.708).

Table 5. Nested-CV tuning: macro-F1 before and after (15 outer runs); p is the corrected paired test. Highlighted = significant gain.

Model	Untuned MF1	Tuned MF1	Gain	p (corr.)
Linear SVM (OvR)	**0.629**	**0.657**	**+0.028**	**<0.001**
Decision tree depth 10	**0.609**	**0.630**	**+0.021**	**0.003**
Nystroem RBF + logistic reg	**0.617**	**0.635**	**+0.018**	**0.002**
SVM RBF	0.621	0.627	+0.005	0.052
MLP 128-64 relu	0.635	0.638	+0.003	0.652
SVM polynomial deg 2	0.626	0.626	-0.000	0.984
Logistic regression (softmax)	0.641	0.639	-0.002	0.695
Random forest	0.640	0.636	-0.003	0.396

Tuning helps models whose defaults were poor: the linear SVM (+0.028; strong regularisation C = 0.01 with balanced weights was selected most often), the depth-10 tree (+0.021; depth 4 with 50 samples per leaf chosen most often, i.e. a much simpler tree) and Nystroem-RBF (+0.018). It does not help the RBF-SVM (+0.005, p = 0.052), polynomial SVM, MLP or logistic regression, and is slightly negative for the random forest (−0.003), where the inner 3-fold search plausibly over-fits its own scoring noise. It is expensive: 4,289 s for the RBF-SVM and 3,237 s for the forest, against 87 s for a single balanced forest that beats every tuned model.

5.4 Per-class performance and significance

Table 6. Per-class precision (P) and recall (R), mean over 15 runs.

Model	P 1	R 1	P 2	R 2	P 3	R 3
Decision tree (raw)	0.538	0.370	0.673	0.801	0.662	0.507
Random forest, balanced	0.556	0.544	0.733	0.777	0.705	0.637
Linear SVM, balanced	0.494	0.628	0.744	0.747	0.701	0.642
MLP 128-64 (reference)	0.609	0.408	0.712	0.832	0.730	0.589

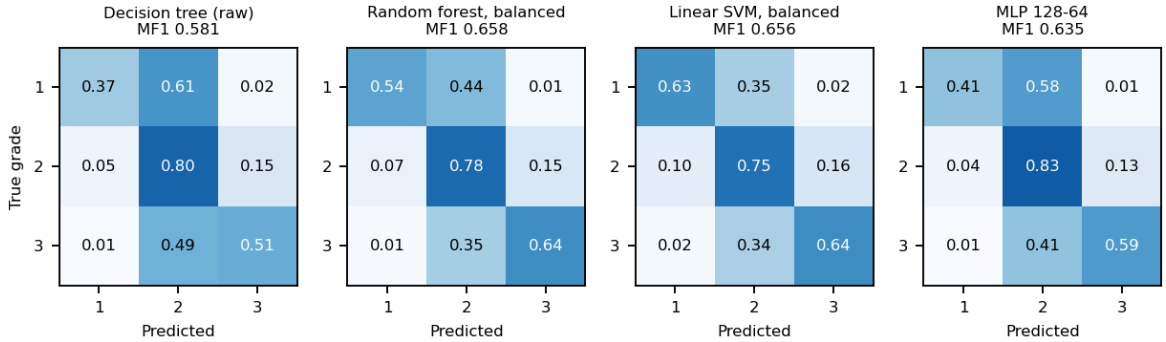


Figure 4. Row-normalised confusion matrices (summed over 15 runs).

Grade 1 is the hard class for every model: the raw tree finds only 37 % of low-damage buildings, the plain models 35–44 %, the balanced forest 54 % and the balanced linear SVM 63 %. Most errors are between adjacent grades (1 ↔ 2 and 2 ↔ 3); grade-1 and grade-3 buildings are rarely confused with each other (at most 2.3 % of grade-1 buildings are labelled grade 3 and at most 2.2 % of grade-3 buildings grade 1 for the four models shown).

Table 7. Selected comparisons on the 20,000-row subsample (15 paired runs, macro-F1). Positive = first method better.

Comparison	Mean diff.	p (paired)	p (corrected)
Balanced random forest vs raw decision tree	+0.077	<0.001	<0.001
Target encoding: SVM-RBF on TE vs raw	+0.141	<0.001	<0.001
Target encoding: tree on TE vs raw	+0.029	<0.001	0.008
Feature extraction: MLP on TE+eng vs raw	+0.113	<0.001	<0.001
Mutual-information top-20 vs none (SVM-RBF)	+0.006	0.004	0.141
PCA 95 % vs none (SVM-RBF)	-0.011	<0.001	<0.001
Balancing: random forest	+0.018	<0.001	<0.001
Balancing: SVM-RBF	+0.012	<0.001	0.035
Balancing: linear SVM	+0.027	<0.001	<0.001
Balancing: logistic regression	+0.000	0.958	0.981
Balanced RF vs balanced linear SVM	+0.002	0.244	0.586
Balanced RF vs balanced SVM-RBF	+0.025	<0.001	<0.001
Balanced RF vs MLP	+0.023	<0.001	0.002
Balanced RF vs decision tree (TE+eng)	+0.048	<0.001	<0.001

6. Confirmation on all 260,601 rows (RQ3)

The methods that scale were rerun on **every** building with stratified 5-fold CV (one repeat, five runs per method). The exact RBF-SVM, polynomial SVMs and the tuning grids need days at this size and stay on the subsample; the Nystroem-RBF + logistic regression stands in for a kernel SVM. Corrected tests have little power with five runs, so p-values are indicative.

Table 8. Full-data results (260,601 rows, 5 runs). MF1 20k = same model on the subsample; gain = full − 20k.

Model	Accuracy	Macro-F1	MF1 20k	Gain	Rec 1
Random forest, balanced	**0.742 ± 0.002**	**0.707 ± 0.003**	**0.658**	**+0.049**	**0.669**
Linear SVM, balanced	0.725 ± 0.002	0.686 ± 0.003	0.656	+0.030	0.694
MLP 128-64, TE+eng (reference)	0.741 ± 0.002	0.686 ± 0.004	0.635	+0.051	0.513
Logistic regression	0.738 ± 0.002	0.674 ± 0.004	0.641	+0.033	0.464
Nystroem-RBF + logistic reg. (kernel approx.)	0.735 ± 0.001	0.671 ± 0.003	–	–	0.469
Logistic regression, balanced	0.699 ± 0.002	0.671 ± 0.002	0.641	+0.029	0.810
Decision tree, TE+eng + Fisher-20	0.733 ± 0.001	0.674 ± 0.004	0.613	+0.062	0.495
Decision tree, TE+eng	0.732 ± 0.002	0.672 ± 0.006	0.609	+0.063	0.483
Decision tree, TE+eng + PCA	0.719 ± 0.001	0.650 ± 0.004	0.588	+0.062	0.441
MLP 128-64, raw (reference)	0.680 ± 0.002	0.611 ± 0.005	0.522	+0.088	0.419
Decision tree, raw (baseline)	0.685 ± 0.001	0.604 ± 0.009	0.581	+0.023	0.356

Table 9. Significance on the full data (5 paired runs).

Comparison	Mean diff.	p (paired)	p (corrected)
Balanced RF vs raw decision tree	+0.103	<0.001	<0.001
Balanced RF vs balanced linear SVM	+0.021	<0.001	<0.001
Balanced RF vs MLP	+0.021	<0.001	<0.001
Balanced RF vs logistic regression	+0.033	<0.001	<0.001
Balanced RF vs decision tree (TE+eng)	+0.035	<0.001	<0.001
Target encoding: tree on TE vs raw	+0.066	<0.001	<0.001
Feature extraction: MLP on TE+eng vs raw	+0.076	<0.001	<0.001

What carries over: target encoding remains essential (tree +0.066, MLP +0.076 versus raw; $p < 0.001$), PCA remains the worst step (tree 0.650 versus 0.672), and the **balanced random forest is again best**, now by a clear margin: macro-F1 0.707, accuracy 0.742, ahead of the balanced linear SVM (+0.021), the MLP (+0.021), logistic regression (+0.033) and the tree (+0.035), all with corrected $p < 0.001$. **What changes with ten times more data:** every model gains 0.02–0.09 macro-F1, most for flexible and under-trained models (MLP +0.051, forest +0.049, tree on TE+eng +0.063) and least for the linear models (+0.03); the MLP now ties the balanced linear SVM (0.686) and beats plain logistic regression, so its earlier deficit was a small-sample effect; and balancing no longer helps logistic regression (0.671 balanced versus 0.674 plain, with accuracy down from 0.738 to 0.699) while it still helps the forest and the linear SVM.

7. Discussion

- **Why target encoding wins.** The geographic ids carry the strongest signal (grade-3 share 8.6 % to 80.8 %) but are nominal labels. Raw integers impose a false order on the SVM, the MLP and the linear models, and one-hot columns for 11,595 regions are mostly empty. Target encoding summarises each region by its local class mix in six dense features. Its hierarchy, smoothing ($\alpha = 20$) and out-of-fold training encoding keep sparse regions from being trusted too much, so training and test encodings have the same noise level. Its benefit grows with data (tree +0.029 on 20k, +0.066 on all rows) as regions contain more buildings.
- **Why PCA fails and selection barely helps.** PCA maximises variance, not class separation, so it discards weak but discriminative directions; MI and Fisher filters remove noise columns, but the RBF-SVM already gives them little weight and the tree ignores them, hence gains within noise.
- **Why class balancing is model-dependent.** Macro-F1 values grade 1 as much as grade 2, whereas accuracy-optimal rules under-predict it. Balancing helps when the base model ranks grade-1 buildings well but is calibrated for the majority (forest, linear SVM, SVM-RBF); for logistic regression recall and precision trade off evenly, so F1 stays flat; at large n flexible models are better calibrated and the benefit shrinks.
- **Why the random forest wins.** Trees split on nominal-like ids and feature interactions without scaling, the ensemble reduces variance, and balanced weights add grade-1 recall (0.54 on 20k, 0.67 on all rows) while keeping grade-2 and grade-3 recall high. It also benefits most from more data. A strongly regularised linear SVM is almost as good on small data because, after encoding, most of the signal is additive; it is also about six times cheaper.
- **Why tuning matters little.** It removes bias from poor defaults (linear SVM, tree) but cannot beat sensible ones, and its inner 3-fold scores are noisy. Choosing the right features and decision rule gave far more than hyper-parameter search.
- **Deep learning as reference.** The MLP is poor on raw features (0.522 / 0.611 on 20k / all rows), gains +0.113 / +0.076 from the same feature extraction, and at best matches the balanced linear SVM on all rows; it never beats the balanced forest ($p < 0.003$ on both scales). On this tabular problem it offers no advantage over classical pipelines and is the least interpretable.

- **Recommendation.** Target encoding + engineered features, then a class-balanced random forest (best, 0.707 macro-F1 / 0.742 accuracy on all rows) or, on small data or tight budgets, a balanced linear SVM (0.656 in 14 s). Always report macro-F1 and per-class recall, and keep every learned step inside the CV fold.

8. Limitations and conclusion

- **Subsample and fixed settings:** the main comparison uses 20,000 of 260,601 rows (Section 6 confirms the key findings on all rows); apart from the eight tuned models, hyper-parameters were fixed a priori. **Winner selection:** "best" is chosen on the same folds it is reported on (slightly optimistic); we therefore test the winner against its rivals. **Dependence:** repeated CV reuses data (corrected tests used); full-data tests rest on five runs. **Scope:** one earthquake, nominal classifiers for an ordinal target, five model families. Explanations in Section 7 are consistent with the data but not isolated by separate experiments.

Conclusion. Fold-safe hierarchical target encoding of the geographic ids is the most important pattern-recognition step (macro-F1 +0.03 to +0.14 for every model family), PCA is harmful, and setting the decision rule for macro-F1 through class balancing is the second lever. Together they lift the best raw baseline from macro-F1 0.581 to 0.658 on 20,000 rows and the final model, a class-balanced random forest, to 0.707 (accuracy 0.742) on all 260,601 rows, significantly ahead of the SVMs, linear models and the MLP reference. **Future work:** exact kernel SVMs on all rows, ordinal-regression losses that respect the order of the grades, tuning of the encoding smoothing, spatial features, and transfer to another earthquake.

References

[1] DrivenData / Kathmandu Living Labs. Richter's Predictor: Modeling Earthquake Damage (data competition), 2019.
[2] Micci-Barreca, D. A preprocessing scheme for high-cardinality categorical attributes. ACM SIGKDD Explorations 3(1), 2001.
[3] Nadeau, C. and Bengio, Y. Inference for the generalization error. Machine Learning 52, 2003.
[4] Breiman, L. Random forests. Machine Learning 45, 2001.
[5] Hastie, T., Tibshirani, R. and Friedman, J. The Elements of Statistical Learning, 2nd ed. Springer, 2009.
[6] Pedregosa, F. et al. Scikit-learn: machine learning in Python. JMLR 12, 2011.

Appendix A. Individual contributions and reproducibility

Work was split into three equal tracks among Sri Harasha Adithya, KVNM Krishnavamsi and Kommi Vishal Venkat Vaibhav; each member presents their own track and commits under their own name.

Member	Track	Responsibilities and personal contribution
[member]	Data, evaluation, baselines	Data loading and EDA; stratified subsample; CV harness; leakage-free pipelines; metrics; raw baselines; integration into main.py; report Sections 1–3. [add personal contribution]
[member]	Feature extraction	Hierarchical out-of-fold target encoding; engineered features; PCA, MI and Fisher selection; ablation; Sections 4.1, 5.2, 7. [add personal contribution]
[member]	Models, balancing, tuning, DL reference	Tree, forest, SVM, linear and MLP families; class balancing; nested tuning; significance tests; full-data run; Sections 4.2–4.3, 5.1, 5.3–5.4, 6. [add personal contribution]

Reproducibility. Python 3.12, scikit-learn 1.6, pandas, NumPy, SciPy, matplotlib (requirements.txt), CPU only. Put train_values.csv and train_labels.csv in data/ and run `python code/main.py` (EDA, tracks, tables, figures; about one hour on 16 cores); `python code/run_ext.py` runs the extended model zoo and nested tuning; `python code/main.py --full` and `python code/full_data.py` run the full-data confirmation. Seed 42 everywhere; finished experiments are cached as JSON in results/. Repository: https://github.com/Krishnavamsi-codes/PR-Project-tbm (public until grades are awarded).